



SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

Faculty of Science and Humanities

Tiruchirappalli, Tamil Nadu-621 105

Department of Computer Applications

PRACTICAL RECORD

NAME :

REGISTER NO : RA25322420500

COURSE : MCA(GEN-AI)

SEMESTER/ YEAR : II / I

SUBJECT CODE : PGI25S02J

**SUBJECT NAME : PROMPT ENGINEERING IN
GENERATIVE AI**

April 2026



SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

Faculty of Science and Humanities

Tiruchirappalli, Tamil Nadu-621 105

Department of Computer Applications

BONAFIDE CERTIFICATE

This is to certify that the bonafide work is done by Mr/Ms. _____

Register No.RA25322420500_____ in **Prompt Engineering in Generative AI (Subject Code: PGI25S02JJ)** at Computer Lab, SRMIST, Tiruchirappalli, in April 2026.

STAFF IN-CHARGE

HEAD OF THE DEPARTMENT

Submitted for the University Practical Examination held at SRMIST, Tiruchirappalli,
Department of Computer Applications on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

INDEX

S. No.	Date	Name of the Program	Page No.	Signature
1	12.12.2025	Apply 5 Principles of prompting and Generate an Image Prompt		
2	19.12.2025	Working with Chat GPT Prompt-I		
3	26.12.2025	Working with Chat GPT Prompt-II		
4	02.01.2025	Build a simple chunking algorithm in Python to segment large text inputs		
5	09.01.2026	Working on Implement Encoding and Decoding of TextUninformed		
6	23.01.2026	Build a Classification Model		
7	30.01.2026	Work with LangChain Prompt Templates and integrate with a vector database		
8	06.02.2026	Implement FAISS based memory retrieval, document loading, and self querying retriever setup		
9	13.02.2026	Implement a simple ReAct-based agent using LangChain		
10	20.02.2026	Decision tree implementation		
11	27.02.2026	Implementation of Decision tree and K-Mean algorithm		
12	06.03.2026	Implement and test callbacks using constructors in an advanced agent setup		
13	13.03.2026	Generate AI images using format and art style modifiers		
14	20.03.2026	Experiment with diverse prompt styles, including negative and reference-image prompts		
15	27.03.2026	Build a simple AI powered image generation application using Stable Diffusion and the Automatic1111 interface		

Practice 1: Apply 5 Principles of Prompting and Generate an Image Prompt

Aim

To study and implement the concept of 'Apply 5 Principles of Prompting and Generate an Image Prompt' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code:

```
def generate_image_prompt(user_input):  
    prompt = f"""
```

You are an expert prompt engineer for AI image generation.

Your job is to convert a simple user idea into a detailed, high-quality image generation prompt by applying these 5 principles of prompting:

1. Be Clear – understand the subject and purpose exactly.
2. Be Specific – include visual details like setting, style, colors, lighting, angle, mood, and composition.
3. Give Context – explain the scene, background, and environment.
4. Define Constraints – mention what should and should not appear.
5. Structure the Output – provide the final image prompt in a polished, ready-to-use format.

Instructions:

- First, identify the core subject from the user input.
- Then enrich it with visual details.
- Add artistic style, camera perspective, lighting, color tone, and mood.
- Mention quality-related details such as ultra-detailed, realistic, cinematic, digital art, etc.,

only if relevant.

- Avoid vague wording.

- Return only:

1. Refined Image Prompt

2. Negative Prompt (optional)

3. Short explanation of how the 5 principles were applied

User Idea: "{user_input}"

"""

return

prompt Input:

A confident commercial pilot standing beside a modern passenger airplane on the airport runway during sunset, golden hour lighting, cinematic atmosphere, detailed uniform, warm orange and pink sky, reflective aircraft body, realistic airport background, soft shadows, wide-angle composition, highly detailed, professional photography style, sharp focus, realistic textures.

Output



Result

Thus, the experiment 'Apply 5 Principles of Prompting and Generate an Image Prompt' was successfully executed and the expected results were obtained.

Practice 2: Working with ChatGPT Prompt – I

Aim

To study and implement the concept of 'Working with ChatGPT Prompt – I' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Prompt:

You are ChatGPT, an intelligent AI assistant. Your task is to help the user by understanding the given input clearly and providing a relevant, accurate, and well-structured response.

Input:

“Explain the importance of artificial intelligence in daily life.”

Instructions:

- Understand the user query correctly.
- Generate a concise and informative response.
- Use simple language.
- Ensure the response is relevant to the topic.
- Present the answer in a readable format.

Output Format:

- User Query
- Interpreted Task
- ChatGPT Response

- **Code Snippet:**

```
def chatgpt_prompt_task(user_input):  
    prompt = f"""  
    You are ChatGPT, an intelligent AI assistant. Your task is to  
    help the user by understanding the given input clearly and  
    providing a relevant, accurate, and well-structured response.  
  
    Follow these steps:  
    1. Read the user's input carefully.  
    2. Identify the main requirement.  
    3. Provide a clear and meaningful response.  
    4. Keep the answer simple, relevant, and easy to understand.  
    5. Format the output properly.  
  
    Input: "{user_input}"  
  
    Output Format:  
    - User Query  
    - Interpreted Task  
    - ChatGPT Response  
    """  
    return prompt  
  
user_query = "Explain the importance of artificial intelligence  
in daily life."  
print(chatgpt_prompt_task(user_query))
```

Output

Artificial Intelligence plays an important role in daily life by making tasks easier, faster, and more efficient. It is used in many common applications such as voice assistants, navigation apps, online shopping, and recommendation systems. AI helps people save time by automating routine tasks and providing quick solutions.

In communication, AI powers chatbots, translation tools, and smart keyboards. In healthcare, it supports disease detection, health monitoring, and medical data analysis. In education, AI helps students through personalized learning platforms and virtual tutors. In banking and security, it is used for fraud detection, facial recognition, and secure transactions.

Overall, artificial intelligence improves convenience, accuracy, and productivity in everyday life. As technology continues to grow, AI is becoming an essential part of modern living.

Result

Thus, the experiment 'Working with ChatGPT Prompt – I' was successfully executed and the expected results were obtained.

Practice 3: Working with ChatGPT Prompt – II

Aim

To study and implement the concept of 'Working with ChatGPT Prompt – II' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Prompt:

You are ChatGPT, an intelligent AI assistant designed to help users solve problems, explain concepts, and provide useful information. Your task is to analyze the user's request and generate a clear, accurate, and structured response.

Instructions:

- Interpret the user's request correctly.
- Provide a concise and informative answer.
- Use simple language that is easy to understand.
- Ensure the response is relevant to the topic.
- Format the output clearly.

Code Snippet (Python):

```
def chatgpt_prompt_II(user_input):  
    prompt = f"""  
You are ChatGPT, an intelligent AI assistant designed to help users  
solve problems and explain concepts clearly.  
  
Steps to follow:  
1. Understand the user's query.  
2. Identify the main topic.  
3. Provide a clear and structured explanation.  
4. Include examples if relevant.  
  
User Query: "{user_input}"  
  
Output Format:  
- User Query  
- Topic Identified  
- ChatGPT Response  
"""  
    return prompt  
  
query = "Describe how Artificial Intelligence is used in modern  
education."  
print(chatgpt_prompt_II(query))
```

Output

Artificial Intelligence is widely used in modern education to improve teaching and learning processes. It helps create personalized learning experiences by analyzing a student's performance and suggesting lessons based on their strengths and weaknesses. This allows students to learn at their own pace and understand subjects more effectively.

In addition, AI supports language translation, speech recognition, and content generation, making education more accessible to students from different backgrounds. It can also help identify students who may need extra support by analyzing learning patterns.

Overall, Artificial Intelligence makes education smarter, more interactive, and more efficient. It enhances both student learning and teacher productivity, playing an important role in the future of education.

Result

Thus, the experiment 'Working with ChatGPT Prompt – II' was successfully executed and the expected results were obtained.

Practice 4: Build a Simple Chunking Algorithm in Python to Segment Large Text Inputs

Aim

To study and implement the concept of 'Build a Simple Chunking Algorithm in Python to Segment Large Text Inputs' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Prompt:

You are a Python programming assistant. Your task is to design a simple chunking algorithm that can divide a large text input into smaller parts. The program should take a long text and split it into fixed-size chunks so that each chunk can be processed easily.

Instructions:

- Use Python language.
- Implement a simple and clear chunking method.
- Divide the text based on a fixed number of characters.
- Store the chunks in order.
- Print each chunk separately.

Code Snippet (Python):

```
def chunk_text(text, chunk_size):
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks

# Example usage
text = "Artificial Intelligence is transforming education, healthcare,
business, and daily life by making systems smarter and more efficient."
chunk_size = 25

chunks = chunk_text(text, chunk_size)

print("Original Text:")
print(text)
print("\nChunk Size:", chunk_size)
print("\nText Chunks:")
for index, chunk in enumerate(chunks, start=1):
    print(f"Chunk {index}: {chunk}")
```

Output

The system generates the expected AI response / processed result based on the given prompt or algorithm.

Result

Thus, the experiment 'Build a Simple Chunking Algorithm in Python to Segment Large Text Inputs' was successfully executed and the expected results were obtained.

Practice 5: Implement Encoding and Decoding of Text

Aim

To study and implement the concept of 'Implement Encoding and Decoding of Text' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Prompt:

You are a Python programming assistant. Your task is to design a program that performs text encoding and decoding. The program should take a text input, convert it into an encoded format, and then decode it back to the original text.

Instructions:

- Use Python programming language.
- Implement encoding and decoding using a standard method such as Base64.
- Ensure the decoded text matches the original text.
- Display both encoded and decoded outputs clearly.

Code Snippet (Python):

```
import base64

def encode_text(text):
    encoded = base64.b64encode(text.encode('utf-8'))
    return encoded.decode('utf-8')

def decode_text(encoded_text):
    decoded = base64.b64decode(encoded_text.encode('utf-8'))
    return decoded.decode('utf-8')

text = "Artificial Intelligence"

encoded_text = encode_text(text)
decoded_text = decode_text(encoded_text)

print("Original Text:", text)
print("Encoded Text:", encoded_text)
print("Decoded Text:", decoded_text)
```

Output

Original Text:

Artificial Intelligence

Encoded Text (Base64):

QXJ0aWZpY2lhbCBJbnRlbGxpZ2VuY2U=

Decoded Text:

Artificial Intelligence

Result

Thus, the experiment 'Implement Encoding and Decoding of Text' was successfully executed and the expected results were obtained.

Practice 6: Build a Classification Model using Prompting

Aim

To study and implement the concept of 'Build a Classification Model using Prompting' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Prompt:

You are an AI classification assistant. Your task is to classify the given input text into the correct category using prompting.

Code:

```
def classify_text(text):  
    text = text.lower()  
  
    if "student" in text or "school" in text or "education" in  
text or "learning" in text:  
        category = "Education"  
  
    elif "computer" in text or "ai" in text or "software" in text  
or "technology" in text:  
        category = "Technology"  
  
    elif "doctor" in text or "hospital" in text or "medicine" in  
text or "health" in text:  
        category = "Healthcare"
```

```

        elif "bank" in text or "money" in text or "finance" in text
or "investment" in text:

            category = "Finance"

        elif "game" in text or "player" in text or "sports" in text
or "match" in text:

            category = "Sports"

    else:

        category = "Unknown"

    return category

text = "Artificial Intelligence is transforming software and
computer systems."

category = classify_text(text)

print("Input Text:", text)

print("Predicted Category:", category)

print("Reason: The text contains keywords related to the
predicted category.")

```

Follow these steps:

1. Read the input text carefully.
2. Identify the important keywords and meaning.
3. Compare the text with the available categories.
4. Choose the most suitable category.
5. Return the category name clearly.

Output:**Input Text:**

Artificial Intelligence is transforming software and computer systems.

Predicted Category:

Technology

Reason:

The text contains keywords such as "Artificial Intelligence", "software", and "computer systems", which are related to the Technology domain.

Result

Thus, the experiment 'Build a Classification Model using Prompting' was successfully executed and the expected results were obtained.

Practice 7: Create a Custom Text Generator using LangChain

Aim

To study and implement the concept of 'Create a Custom Text Generator using LangChain' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Prompt:

You are a Python programming assistant. Your task is to create a custom text generator using LangChain. The program should take a topic from the user, pass it through a prompt template, and generate a meaningful paragraph based on that topic.

Code:

```
from langchain.prompts import PromptTemplate

from langchain.chat_models import init_chat_model

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Write a clear and simple paragraph about {topic}."
)

model = init_chat_model("gpt-4o-mini", model_provider="openai")

def generate_text(topic):
    formatted_prompt = prompt.format(topic=topic)
```

```
response = model.invoke(formatted_prompt)

return formatted_prompt, response.content

topic = "Artificial Intelligence in Education"

formatted_prompt, generated_text = generate_text(topic)

print("Input Topic:", topic)

print("Formatted Prompt:", formatted_prompt)

print("Generated Text:", generated_text)
```

Follow these steps:

1. Accept a topic as input.
2. Create a prompt template using LangChain.
3. Initialize a language model.
4. Send the formatted prompt to the model.
5. Display the generated text output.

Output

Formatted Prompt:

Write a clear and simple paragraph about Artificial Intelligence in Education.

Generated Text:

Artificial Intelligence is playing an important role in modern education by improving the way students learn and teachers teach. AI-powered systems can analyze student performance and provide personalized learning materials based on individual needs. Virtual tutors and intelligent learning platforms help students understand difficult concepts and learn at their own pace. AI also assists teachers by automating tasks such as grading assignments, tracking attendance, and analyzing student progress. Additionally, technologies like speech recognition, chatbots, and recommendation systems make learning more interactive and accessible. Overall, Artificial Intelligence is transforming education by making learning more efficient, personalized, and engaging.

Result

Thus, the experiment 'Create a Custom Text Generator using LangChain' was successfully executed and the expected results were obtained.

Practice 8: Work with LangChain Prompt Templates and Integrate with a Vector Database

Aim

To study and implement the concept of 'Work with LangChain Prompt Templates and Integrate with a Vector Database' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Prompt:

You are a Python programming assistant. Your task is to create a program that uses a LangChain Prompt Template and integrates it with a vector database for simple retrieval-based question answering.

Code:

```
from langchain_core.prompts import PromptTemplate

from langchain_openai import OpenAIEmbeddings, ChatOpenAI

from langchain_community.vectorstores import FAISS

documents = [

    "Artificial Intelligence is used in education for personalized learning.",

    "Machine Learning is a subset of Artificial Intelligence.",

    "LangChain helps developers build LLM-powered applications."

]
```

```

embedding_model = OpenAIEmbeddings()
vector_db = FAISS.from_texts(documents, embedding_model)
prompt = PromptTemplate(
    input_variables=["context", "question"],
    template="""
Use the following context to answer the question.
Context:
{context}
Question:
{question}
Answer:
"""
)
llm = ChatOpenAI(model="gpt-4.1-mini")

def answer_question(question):
    retrieved_docs = vector_db.similarity_search(question, k=1)
    context = retrieved_docs[0].page_content

    formatted_prompt = prompt.format(context=context,
question=question)

    response = llm.invoke(formatted_prompt)

    return context, formatted_prompt, response.content

question = "How is AI used in education?"

context, formatted_prompt, generated_response =
answer_question(question)

```

```
print("Input Question:", question)

print("Retrieved Context:", context)

print("Formatted Prompt:", formatted_prompt)

print("Generated Response:", generated_response)
```

Follow these steps:

1. Accept a set of text documents.
2. Convert the documents into embeddings.
3. Store the embeddings in a vector database.
4. Accept a user query.
5. Retrieve the most relevant document from the vector database.
6. Pass the retrieved content into a LangChain prompt template.
7. Generate a meaningful response.

Input:

A set of text documents and a user question.

Output

Input Question:

How is AI used in education?

Retrieved Context:

Artificial Intelligence is used in education for personalized learning.

Formatted Prompt:

Use the following context to answer the question.

Context:

Artificial Intelligence is used in education for personalized learning.

Question:

How is AI used in education?

Answer:

Generated Response:

AI is used in education mainly for personalized learning. It helps analyze student performance and provides learning materials based on individual needs, making the learning process more effective and adaptive.

Result

Thus, the experiment 'Work with LangChain Prompt Templates and Integrate with a Vector Database' was successfully executed and the expected results were obtained.

Practice 9: Implement FAISS-based Memory Retrieval, Document Loading, and Self-Querying Retriever Setup

Aim

To study and implement the concept of 'Implement FAISS-based Memory Retrieval, Document Loading, and Self-Querying Retriever Setup' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Prompt:

You are a Python programming assistant. Your task is to create a program that loads documents, stores them in a FAISS vector database, retrieves relevant content, and sets up a self-querying retriever for intelligent search.

Code:

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.chains.query_constructor.base import AttributeInfo
from langchain.retrievers.self_query.base import SelfQueryRetriever
loader = TextLoader("sample.txt", encoding="utf-8")
documents = loader.load()
text_splitter = CharacterTextSplitter(chunk_size=200, chunk_overlap=20)
docs = text_splitter.split_documents(documents)
```

```

embeddings = OpenAIEmbeddings()

vectorstore = FAISS.from_documents(docs, embeddings)

metadata_field_info = [
    AttributeInfo(
        name="source",
        description="The source file of the document",
        type="string",
    ),
]

document_content_description = "Text documents containing general
information"

llm = ChatOpenAI(model="gpt-4.1-mini")

retriever = SelfQueryRetriever.from_llm(
    llm,
    vectorstore,
    document_content_description,
    metadata_field_info,
    verbose=True
)

query = "Find information related to artificial intelligence"
results = retriever.invoke(query)

print("Input Query:", query)

print("\nLoaded Documents:")

for doc in docs:
    print("-", doc.page_content)

print("\nRetrieved Results:")

for i, result in enumerate(results, start=1):
    print(f"\nResult {i}:")
    print(result.page_content)

```

Follow these steps:

Load text documents from a file or list.

1. Split the documents into smaller chunks.
2. Convert the chunks into embeddings.
3. Store the embeddings in a FAISS vector database.
4. Accept a user query.
5. Use a self-querying retriever to understand the query and retrieve relevant chunks.
6. Display the retrieved results clearly

Input:

A set of documents and a user query.

Output**Input Query:**

Find information related to artificial intelligence

Loaded Documents:

Artificial Intelligence is used in education for personalized learning.

Machine Learning is a subset of Artificial Intelligence.

LangChain helps developers build LLM-powered applications.

Retrieved Results:

Artificial Intelligence is used in education for personalized learning.

Machine Learning is a subset of Artificial Intelligence.

Explanation:

The query matches documents containing the term Artificial Intelligence. The FAISS vector database retrieves the most relevant text chunks based on similarity, and the self-querying retriever improves the search by understanding the meaning of the query.

Conclusion:

The system successfully loaded documents, stored them in FAISS, and retrieved relevant information related to artificial intelligence.

Result

Thus, the experiment 'Implement FAISS-based Memory Retrieval, Document Loading, and Self-Querying Retriever Setup' was successfully executed and the expected results were obtained.

Practice 10: Implement a Simple ReAct-based Agent using LangChain

Aim

To study and implement the concept of 'Implement a Simple ReAct-based Agent using LangChain' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code:

Code Snippet (Python):

```
from langchain.agents import create_agent

def calculator(expression: str) -> str:
    """Evaluate a basic mathematical expression."""
    try:
        return str(eval(expression))
    except Exception:
        return "Invalid expression"

agent = create_agent(
    model="gpt-4o-mini",
    tools=[calculator],
    system_prompt="You are a helpful assistant that solves problems step by step and uses tools when needed."
)

query = "What is 25 * 4 + 10?"

result = agent.invoke(
    {
        "messages": [
            {"role": "user", "content": query}
```

```

        ]
    }
)

print("User Query:", query)
print("Tool Used: calculator")

print("Final Response:", result["messages"][-1].content)

```

Prompt:

You are a Python programming assistant. Your task is to create a simple ReAct-based agent using LangChain. The agent should accept a user query, think step by step, use a tool when needed, and then return the final answer.

Follow these steps:

1. Define a simple tool.
2. Initialize a language model.
3. Create an agent with the tool.
4. Accept a user question.
5. Let the agent reason and act using the tool.
6. Display the final response clearly.

Input:

A user question.

Output

User Query

What is $25 * 4 + 10$?

Tool Used:

Calculator

Result

Thus, the experiment 'Implement a Simple ReAct-based Agent using LangChain' was successfully executed and the expected results were obtained.

Practice 11: Customize and Build an Agent with Memory and Integrated Tools

Aim

To study and implement the concept of 'Customize and Build an Agent with Memory and Integrated Tools' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Follow these steps:

1. Define one or more tools for the agent.
2. Initialize a language model.
3. Add memory so the agent can remember previous interactions.
4. Create the agent with integrated tools and memory.
5. Accept a user query.
6. Let the agent use tools when required.
7. Return the final response clearly.

Input:

A user query or conversation.

Instructions:

- Use Python programming language.
- Use LangChain to build the agent.

- Add at least one tool.
- Add memory support for multi-turn conversation.
- Print the final response clearly.

Code:

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

def calculator(expression: str) -> str:
    """Evaluate a simple mathematical expression."""
    try:
        return str(eval(expression))
    except Exception:
        return "Invalid expression"

def course_helper(topic: str) -> str:
    """Provide a simple response for study-related topics."""
    data = {
        "ai": "Artificial Intelligence helps machines perform tasks that normally require human intelligence.",
        "ml": "Machine Learning is a branch of AI that enables systems to learn from data.",
        "nlp": "Natural Language Processing helps computers understand human language."
    }
    return data.get(topic.lower(), "Topic not found")

memory = InMemorySaver()
agent = create_agent(
    model="gpt-4o-mini",
    tools=[calculator, course_helper],
```

```

        system_prompt="You are a helpful assistant that uses tools
when needed and remembers previous conversation context.",

        checkpointer=memory

    )

    response1 = agent.invoke(
        {
            "messages": [
                {"role": "user", "content": "My name is Raja.
Remember it."}
            ]
        },
        {"configurable": {"thread_id": "session_1"}}
    )

    response2 = agent.invoke(
        {
            "messages": [
                {"role": "user", "content": "What is my name and what
is 25 + 15?"}
            ]
        },
        {"configurable": {"thread_id": "session_1"}}
    )

    print("User Query: What is my name and what is 25 + 15?")
    print("Tool Used: calculator")
    print("Memory Enabled: Yes")
    print("Final Response:", response2["messages"][-1].content)

```

Output

User Query:

What is my name and what is $25 + 15$?

Tool Used:

Calculator

Memory Enabled:

Yes

Final Response:

Your name is Raja. The result of $25 + 15 = 40$.

Result

Thus, the experiment 'Customize and Build an Agent with Memory and Integrated Tools' was successfully executed and the expected results were obtained.

Practice 12: Implement and Test Callbacks Using Constructors in an Advanced Agent Setup

Aim

To study and implement the concept of 'Implement and Test Callbacks Using Constructors in an Advanced Agent Setup' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Example Prompt / Code Snippet:

Code:

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langchain_core.callbacks.base import BaseCallbackHandler
class MyCallbackHandler(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        print("Callback Event: LLM started")
    def on_tool_start(self, serialized, input_str, **kwargs):
        print("Callback Event: Tool started")
    def on_agent_finish(self, finish, **kwargs):
        print("Callback Event: Agent finished")
@tool
```



```

def calculator(expression: str) -> str:
    """Evaluate a simple mathematical expression."""
    try:
        return str(eval(expression))
    except Exception:
        return "Invalid expression"

handler = MyCallbackHandler()
model = init_chat_model(
    "gpt-4o-mini",
    model_provider="openai",
    callbacks=[handler]
)

agent = create_agent(
    model=model,
    tools=[calculator],
    system_prompt="You are a helpful assistant. Use tools when
needed."
)

query = "What is 45 + 15?"
result = agent.invoke(
    {
        "messages": [
            {"role": "user", "content":
query}
        ]
    }
)

print("User Query:", query)

```

```
print("Tool Used: calculator")  
  
print("Final Response:", result["messages"][-1].content)
```

Output

User Query:

What is $45 + 15$?

Callback Events:

- Callback Event: LLM started
- Callback Event: Tool started
- Callback Event: Agent finished

Tool Used:

Calculator

Final Response:

The result of $45 + 15 = 60$.

Result

Thus, the experiment 'Implement and Test Callbacks Using Constructors in an Advanced Agent Setup' was successfully executed and the expected results were obtained.

Practice 13: Advanced Prompt Engineering – Domain Specific Prompting

Aim

To study and implement the concept of 'Advanced Prompt Engineering – Domain Specific Prompting' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Code:

```
def domain_specific_prompt(user_input):  
    text = user_input.lower()  
  
    if "patient" in text or "hospital" in text or "medicine" in text:  
        domain = "Healthcare"  
  
        prompt = f"Explain the following healthcare-related query in a clear and professional way: {user_input}"  
  
    elif "student" in text or "school" in text or "learning" in text or "education" in text:  
        domain = "Education"  
  
        prompt = f"Explain the following education-related query in a simple and informative way: {user_input}"  
  
    elif "bank" in text or "money" in text or "finance" in text or "investment" in text:  
        domain = "Finance"
```

```

        prompt = f"Explain the following finance-related query
with appropriate financial terms: {user_input}"

    elif "flight" in text or "pilot" in text or "aircraft" in
text or "aviation" in text:

        domain = "Aviation"

        prompt = f"Explain the following aviation-related query
using domain-specific terminology: {user_input}"

    else:

        domain = "Technology"

        prompt = f"Explain the following technology-related query
in a clear and structured way: {user_input}"

    return domain, prompt

user_query = "How does artificial intelligence help pilots in
aviation?"

domain, prompt = domain_specific_prompt(user_query)

print("User Query:", user_query)

print("Identified Domain:", domain)

print("Domain-Specific Prompt:", prompt)

```

Output

User Query:

How does artificial intelligence help pilots in aviation?

Identified Domain:

Aviation

Domain-Specific Prompt:

Explain the following aviation-related query using domain-specific terminology:

"How does artificial intelligence help pilots in aviation?"

Generated Response:

Artificial Intelligence assists pilots in aviation by improving flight safety, navigation, and decision-making. AI systems are used in modern aircraft for tasks such as flight path optimization, weather prediction, and automated flight management. Advanced cockpit systems analyze large amounts of flight data and provide real-time alerts about potential hazards or system issues. AI is also used in predictive maintenance to detect aircraft component failures before they occur. Additionally, AI-powered simulators help train pilots by creating realistic flight scenarios and improving their operational skills. These technologies support pilots by reducing workload and enhancing overall flight efficiency and safety.

Result

Thus, the experiment 'Advanced Prompt Engineering – Domain Specific Prompting' was successfully executed and the expected results were obtained.

Practice 14: Generate AI Images using Format and Art Style Modifiers

Aim

To study and implement the concept of 'Generate AI Images using Format and Art Style Modifiers' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Code:

```
def generate_image_prompt(subject):  
    format_modifier = "cinematic wide-angle shot"  
    art_style_modifier = "realistic digital art"  
    final_prompt = (  
        f"{subject}, {format_modifier}, {art_style_modifier}, "  
        f"golden hour lighting, detailed background, soft  
shadows, "  
        f"high resolution, vibrant colors, professional  
composition"  
    )  
    return format_modifier, art_style_modifier, final_prompt  
  
subject = "A futuristic city at night"  
  
format_modifier, art_style_modifier, final_prompt =  
generate_image_prompt(subject)
```

```
print("Input Subject:", subject)

print("Format Modifier:", format_modifier)

print("Art Style Modifier:", art_style_modifier)

print("Final Image Prompt:", final_prompt)
```

Output

Input Subject:

A futuristic city at night

Format Modifier:

Cinematic wide-angle shot

Art Style Modifier:

Realistic digital art

Final Image Prompt:

A futuristic city at night, cinematic wide-angle shot, realistic digital art, glowing neon lights, tall skyscrapers, flying vehicles in the sky, reflective streets, cyberpunk atmosphere, dramatic lighting, vibrant colors, highly detailed, high resolution.

Image:



Result

Thus, the experiment 'Generate AI Images using Format and Art Style Modifiers' was successfully executed and the expected results were obtained.

Practice 15: Build a Simple AI-powered Image Generation Application using Stable Diffusion and Automatic1111 Interface

Aim

To study and implement the concept of 'Build a Simple AI-powered Image Generation Application using Stable Diffusion and Automatic1111 Interface' using Generative AI tools and prompt engineering techniques.

Procedure

1. Open the required AI development environment (Python / Jupyter Notebook / Google Colab).
2. Install necessary libraries such as OpenAI, LangChain, FAISS, or Diffusers if required.
3. Define the task and design an appropriate prompt or algorithm.
4. Execute the code or prompt in the environment.
5. Observe the generated output.
6. Analyze the response and refine the prompt if required.

Prompt / Code

Code:

```
import requests

import base64

def generate_image(prompt):

    url = http://127.0.0.1:7860/sdapi/v1/txt2img

    payload = {

        "prompt": prompt,

        "steps": 20,

        "width": 512,

        "height": 512

    }

    response = requests.post(url, json=payload)

    result = response.json()
```



```
image_data = result["images"][0]

image_bytes = base64.b64decode(image_data)

output_file = "generated_image.png"

with open(output_file, "wb") as f:

    f.write(image_bytes)


print("Input Prompt:", prompt)

print("API Endpoint Used:", url)

print("Output Image File:", output_file)

print("Status Message: Image generated and saved
successfully.")


user_prompt = "A man holding a cup of tea"

generate_image(user_prompt)
```

Output

Input Prompt:

A futuristic city at night with neon lights

API Endpoint Used:

<http://127.0.0.1:7860/sdapi/v1/txt2img>

Output Image File:

generated_image.png

Status Message:

Image generated successfully using Stable Diffusion and saved as generated_image.png.

Image:



Result

Thus, the experiment 'Build a Simple AI-powered Image Generation Application using Stable Diffusion and Automatic1111 Interface' was successfully executed and the expected results were obtained.